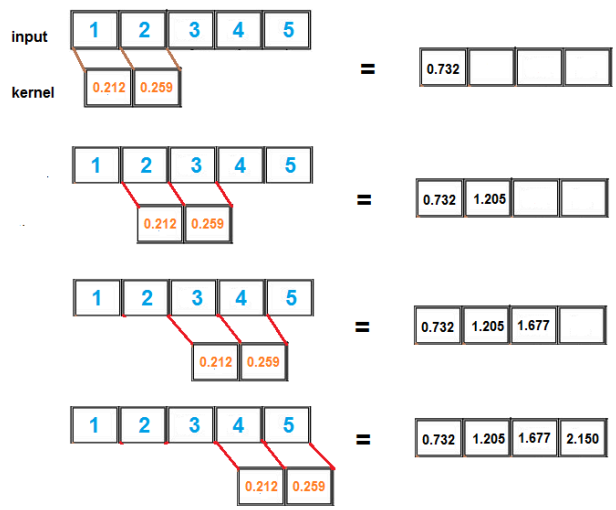
Convolutional Neural Network (CNNs):

What is convolution?

It's a method to combine two different ordered sets into one. It can be thought of as sliding one flipped ordered set over the other and taking the point wise multiplication of **overlapping elements** and summing them up to form an element of the output. So, for discrete ordered set, it would look like:

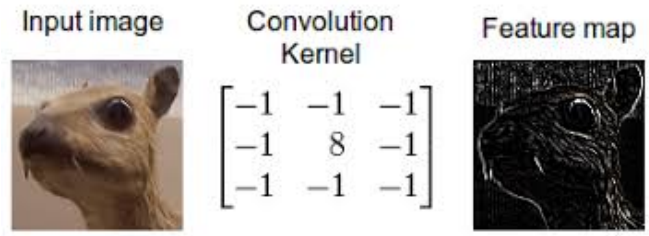
$$[X \star Y]_s = \sum_{i=1}^{n} X(i) * Y(s-i)$$

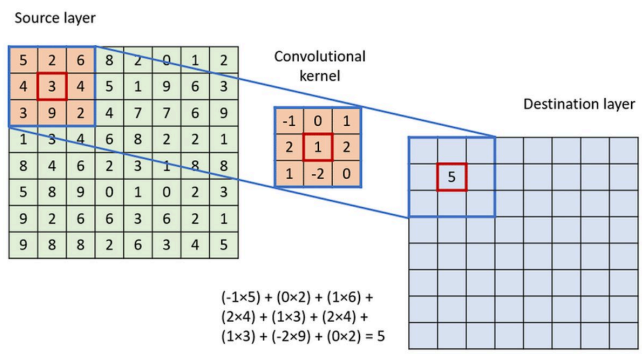


Where X and Y are two ordered sets, s is the sth variable of the output. For continuous case where X and Y are functions of some variable t, it would look like:

$$[X \star Y](s) = \int_{-\infty}^{\infty} X(t) * Y(s-t)dt$$

In image processing, convolution is used to apply "kernels" or "filters" over the images. Filters are small matrices. This matrix is passed over the entire image, multiplying the overlapping elements and summing them up and generating new elements for the next part of the image this way.





Convolution as an operation in neural networks

Technically, the operations we do for each node can individually be thought of as single element set convolution. In CNNs, the nodes are replaced by **filters**, almost. Suppose there exist 64 kernels in-between two layers. Each individual kernel will be convoluted on the input image, forming its own **"feature map"** as output. The output of this layer would be 64 feature maps. This is known as the depth of the image. If the image is RGB, the depth of the image would be 3 and the filters for the first layer have to be some $i * j * 3$. In our previous case, the filters of the second layer would have to have the depth of 64. The kernels can be considered as a set of weights themselves where each element of the kernel is a weight. This weight is learnt the same way as any weight is learnt in traditional ANNs, by **back propagation**.

> Side note: There's some speciality in CNN back propagation as calculating the gradient itself involves taking the convolution of the flipped set of error values and the weights.

Some key pitfalls in CNN

1. **Overfitting the data hunger**: because of the high number of parameters, CNNs tend to memorise the data rather than learn it. Data augmentation is an easy fix for that.
2. **Vanishing gradients**: Your error step is based on all the computations that happen upstream. Suppose there is some function inbetween like $tanh$. Now, for big values, this function flat lines. This means the gradient of it at those high values is close to 0 because of the **flatness**. This is not desirable as it pushed our error values towards 0. Which

means that the first few layers basically wouldn't be getting much changes at all. This doesn't only happen with activation function, but the entire loss function too. One can think of gradient vanishing like a blind man figuring out how to get down from the mountain safely when the mountain top itself is flat. So, the blind guy feels like he's already reached the bottom of the mountain. A good way to solve this is to have some function like ReLU as the activation function. Another way is to just bypass the layer if the layer is doing nothing but dampen the errors. Vanishing gradients are also can be caused by having improper initial weights. As your error propagation is dependent on the upstream weights, if they themselves are closer to 0, so will the passing error will be. Use some good weight initiation technique. There are also techniques like **gradient clipping** and **batch normalisation**.

3. **Spatial information loss**: this is due to down sampling. This can be solved by having strided convolutions instead of heavy pooling.
4. **Adversarial vulnerability**: CNNs can easily be fooled by planted noise because they rely way too much on high frequency textures rather than global shapes.
5. **Rotational equivalence**: CNNs by design are translation invariant. As in, even if we move the cat image in x-y directions, it will still be able to detect the cat. But, this is not the case for rotations.

How are CNNs used in robotics:

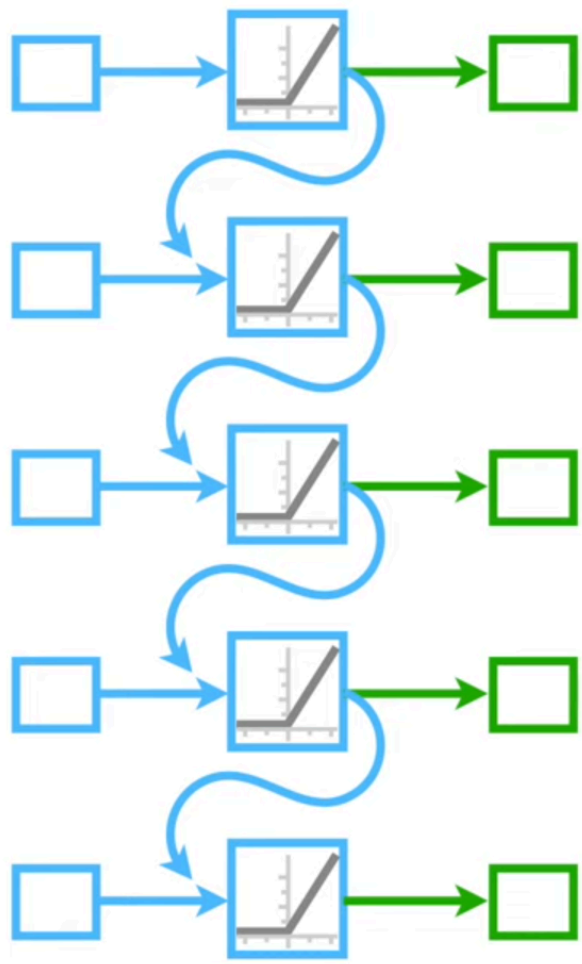
Depth maps and height maps are ideal to be processed by said CNNs, which is made to fit some transformation function which basically "flattens" an image or grid like data into a **meaningful vector** of size n. This vector, along with the proprioception data, is given to some other model. As you saw earlier on how RGB can represent 3 channels of data, same can be done for **temporal coherence**. We can treat time as another dimension in channels. Boston dynamics and ETH Zurich use CNNs like **terrain analysers** which transform the terrain data into a vector embedding.

Now we know how to make sense out of image data for our system. But how do we now use this vector embedding?
We want some model which takes advantage of temporal coherence, because our robots are hardly ever "YOLO" based. In earlier days, this was done by using Recurent Neural Networks (RNNs).

Recurent Neural networks (RNNs):

Recurrence for temporal coherence

RNNs aren't much different from ANNs. The only difference is that the previous data is fed to the input by summing the incoming input to the node and the old output of the node. This can be thought of as having two copies of some ANN where the output of the top ANN gets fed into the input of the bottom ANN by taking a weighted sum on the ANN-1 output and ANN-2 input. The weights for both of these ANNs is the same, because they are the same ANNs. Obviously, you can have multiple layers of this which is what happens usually as shown in the diagram below.



Common pitfalls with RNNs:

The most common problem is the same vanishing/exploding gradient problem we had with the CNNs. But it is more critical here. Suppose the weight is above 1 for summing the input and output and we are retaining the last 5 values. Now, the oldest value would pass through the first one, then get multiplied by the weight, and so on till it reaches the final ANN. At this point, the term would look like $i_0 * w^5 + i_1 * w^4 + i_2 * w^3 + i_3 * w^2 + i_4 * w^1$. Usually, the RNNs have over 50 such ANN copies. This would mean that the gradient we take on this for back propagation will be very high because of the powers on the weights. Meaning that, we'll be taking very huge, varying steps. So, basically, we'll be just bouncing around all over the place.

If we make the weights be less than 1, the gradient just reaches 0 due to the powers on the weights.

How do we solve this? By using Long Short Term Memory (LSTM).

Long Short Term Memory

Till now, the RNN used simple weights to feed forward the data to the network, but obviously, that had its issues. We also can't use the same old Bellman equation with some γ because then we'd have to calculate the gradient of an infinite sequence (somewhat). So, we come up with a new system called LSTM. This mainly has three parts, forget gate, input gate and output gate.

Before diving into this, imagine that there are two channels. A long term channel which holds information that has to be kept throughout some portion of the episode and short term channel which represents the immediate output of the LSTM at that time step.

Forget gate takes the input and the short term memory (I'll just call these STM and LTM), and passes it though sigmoid function which squishes the value between 0 and 1. This value basically represents how much LTM should be scaled by or how much of LTM should be remembered before the next step. The higher the product of input and STM, higher this value. This is multiplied with LTM and passed to the next layer.

Input gate takes two weighted sums with different weights of input and STM and passes one through sigmoid and one through $tanh$. These values are multiplied together and added to the updated LTM from last layer. This is the output of LTM. sigmoid part determines how much of the $tanh$ should be added and $tanh$ determines what should be added. The weights for all of these weighted sums are learnt in the traditional ANN way.

Output gate applies another weighted sum on STM and input and passes it through sigmoid, which is then multiplied with $tanh$ of updated LTM. This is the output of STM for this layer

Now, to get recurrence, these channels are connected from one layer to another and input for each layer is the temporal data that we want to give the model. For example, suppose we are giving the model last 3 data points. First layer would get oldest data, second layer will get middle value as input and 3 layer would get the latest data.

This seems computationally heavy. Gated Recurrent unit or GRU combines input and forget layer into a single update layer to reduce computational complexity. But nowadays, transformers are replacing LSTMs by using "Attention" instead of these gates.

Transformers and the Attention Mechanism

The Shift from Recurrence to Parallelism

LSTMs and RNNs process data sequentially, which creates a "history bottleneck." To know the state at time t, you must compute all states from 0 to $t-1$. Transformers replace this "looping" architecture with a **Parallel** architecture. Instead of a hidden state that carries memory forward, Transformers use **Attention** to look at the entire sequence at once.

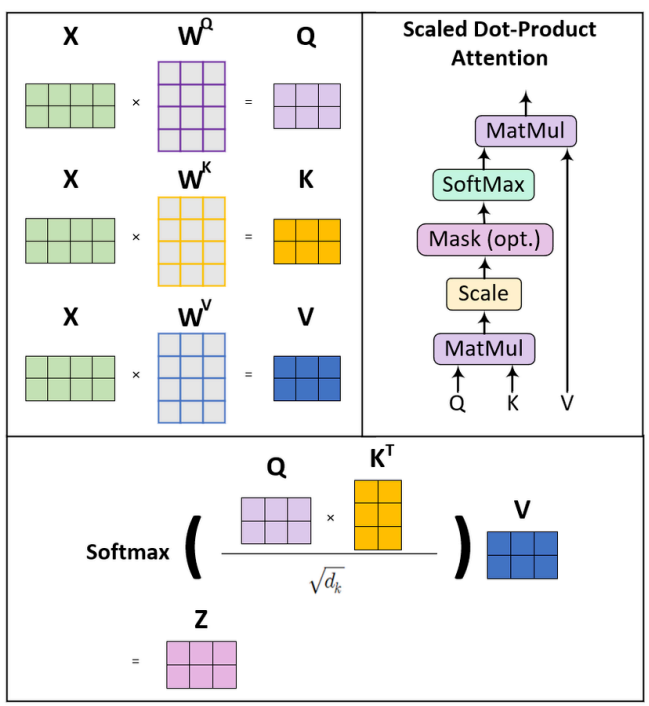
The Self-Attention Mechanism (Q, K, V)

The core of the Transformer is "Scaled Dot-Product Attention." Imagine every piece of data in your sequence (a word, a heightmap slice, or a robot's joint state) is transformed into three different vectors:

1. **Query (Q):** What this specific time step is "looking for" in the rest of the sequence.
2. **Key (K):** A description of what information this time step "contains."
3. **Value (V):** The actual data/features that will be passed on if this step is found relevant.

The "Attention" is calculated by taking the dot product of the Query (Q) with all available Keys (K). This tells the model how much "attention" to pay to every other part of the sequence.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



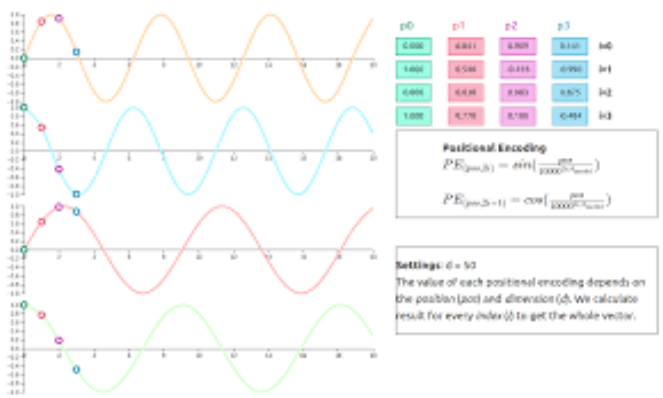
> **Side note:** The division by $\sqrt{d_k}$ (the dimension of the keys) is a scaling factor to prevent the dot products from growing too large, which would push the softmax into regions with extremely small gradients (vanishing gradients again!).

Multi-Head Attention

Instead of performing a single attention calculation, Transformers use **Multi-Head Attention**. This is like having 8 or 16 different "perspectives" looking at the data at once. One "head" might focus on the immediate previous step (short-term momentum), while another head might focus on a specific visual landmark from 5 seconds ago (long-term spatial context). Obviously, due to the "black box nature" of ANNs, we can't determine this and I remember reading a few different studies which suggest that this information is usually learnt to be cross-correlated for tight packing of facts.

Positional Encoding

Because Transformers process all data points simultaneously, they have no inherent sense of "order." To a Transformer, a sequence and a shuffled version of that sequence look the same. To fix this, we add **Positional Encodings** to the input embeddings. These are unique mathematical signals (often sine and cosine waves of different frequencies) that "tag" each data point with its position in time.



Why Transformers are replacing LSTMs in Robotics:

1. **Perfect Long-Term Memory:** Unlike LSTMs, where information can be "washed out" by forget gates, a Transformer has a direct mathematical link to every previous time step. If a bipedal robot slipped 3 seconds ago, it can look back at that exact frame with zero loss in resolution.
2. **Parallel Training:** Since there are no loops, we can use massive GPU clusters to train on years of simulated locomotion data in a fraction of the time it would take an LSTM.

3. **Global Context:** In terrain analysis, the robot can simultaneously attend to the texture under its current foot and the obstacle 5 meters away, integrating this into a single "spatial-temporal" embedding.

Common Pitfalls and Trade-offs:

1. **Quadratic Complexity ($O(n^2)$):** The memory cost of Attention grows with the square of the sequence length. If you double the history the robot looks at, the computation cost quadruples. This makes long-term history expensive for embedded hardware.
2. **The KV Cache:** During real-time "inference" (running the robot), the model has to store the Keys and Values of all past steps in a "KV Cache." This can consume significant VRAM on edge devices like an NVIDIA Jetson.
3. **Data Hunger:** Transformers generally have fewer inductive biases than CNNs or RNNs, meaning they require much more data to "figure out" the rules of the world from scratch.

Vision Transformers (ViTs)

Interestingly, the same Attention mechanism is now being used to replace CNNs. In a **Vision Transformer**, an image is broken into "patches" (like a grid), and each patch is treated as a token in a sequence. The model then uses Self-Attention to see how different parts of the image relate to each other globally, often outperforming CNNs on complex recognition tasks.

Hierarchical Transformers

Solving the Scaling Problem

The primary limitation of the "Vanilla" Transformer (like the original ViT) is its **Global Attention** mechanism. Because every token attends to every other token, the computational cost is $O(n^2)$. This makes it impossible to process high-resolution images or long-duration robot sensor logs. **Hierarchical Transformers** solve this by re-introducing a pyramid-like structure similar to CNNs, but maintaining the flexibility of Attention.

Architectural Philosophy: The Pyramid

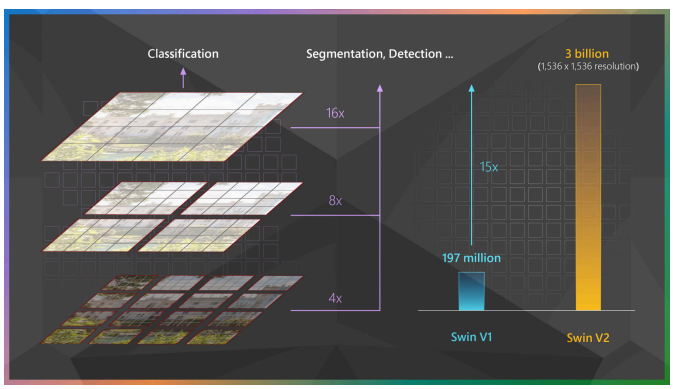
Unlike standard Transformers that maintain a constant resolution (e.g., 16×16 patches) throughout the entire network, Hierarchical Transformers reduce the number of tokens as the network gets deeper while increasing the feature dimension (channel depth).

1. **Stage 1 (Local):** The model starts with very fine-grained patches (e.g., 4×4 pixels). Attention is only calculated within small, local windows.
2. **Stage 2 (Merging):** Neighboring patches are merged into a single token (e.g., 8×8 area).
3. **Stage 3 (Global):** Eventually, the resolution is low enough that the model can afford to perform global attention across the entire feature map.

The Swin Transformer (Shifted Windows)

The most famous Hierarchical Transformer is the **Swin (Shifted Window) Transformer**. It solves the "boundary problem" that occurs when you only look at local windows.

- **Windowed Multi-Head Self-Attention (W-MSA):** In the first layer, the image is divided into non-overlapping windows. Attention is only computed *inside* each window. This is $O(n)$ complexity (Linear).
- **Shifted Windowing (SW-MSA):** In the next layer, the window boundaries are shifted by half a window size. This "breaks" the previous boundaries, allowing information to flow between the neighbouring sections.



> **Side note:** Mathematically, the shift ensures that "cross-window" connections are made without ever actually needing to compute a massive $O(n^2)$ global matrix.

Hierarchical vs. Flat: Why it Matters for Locomotion

In your bipedal RL project, a hierarchical approach allows for **Multi-Scale Temporal/Spatial Processing**:

- **High Resolution:** The model can "see" small cracks or pebbles in the heightmap (early layers, small windows).
- **Abstract Concepts:** The model can understand the "Global Slope" or "Target Goal" (deep layers, merged tokens).

- **Efficiency:** Because of the $O(n)$ scaling, you can feed in a much higher-frequency sensor stream (e.g., 1000Hz IMU data) without the controller's latency exploding.

Key Advantages over Vanilla Transformers:

- **Efficiency:** $O(n)$ complexity allows for high-resolution input (1024x1024 images are feasible).
- **Inductive Bias:** By forcing local attention first, the model learns "spatial locality" naturally, making it easier to train on smaller datasets than a flat ViT.
- **Dense Prediction:** Because it maintains high resolution in early layers, it is much better for tasks like **Semantic Segmentation** (labeling every pixel) which is crucial for a robot identifying "safe-to-step" areas.

Pitfalls:

1. **Complexity of Implementation:** Shifted window math and patch merging are significantly harder to implement in a custom kernel compared to the simple matrix multiplication of a flat Transformer.
2. **Fixed Window Size:** If an important feature is larger than the window size in early layers, the model might struggle to capture it until much deeper in the network.

How is this useful for us:

For cases like our bipedal locomotion, this structure is very useful. We can divide the work as:

- **Low level layers**: Focus on the high frequency jitter in induvidual motor encoders
- **Mid level layers**: Focus on controlling an entire limb
- **High level layers**: Focus on global trajectory of the robot.